

CompSci 330: Design and Analysis of Algorithms

Summer 2024, Homework 1

Due: 7/10/2024 @ 11:59pm

Directions

How to Do Homework. We recommend the following three step process for homework to help you learn and prepare for exams.

1. Give yourself 15-20 minutes per problem to try to solve on your own, without help or external materials, as if you were taking an exam. Try to brainstorm and sketch the algorithm for applied problems. Don't try to type anything yet.
2. After a break, review your answers. Lookup resources or get help (from peers, office hours, Ed discussion, etc.) about problems you weren't sure about.
3. Rework the problems, fill in the details, and typeset your final solutions.

Submission. Your written solutions should be typeset and submitted as a single PDF on Gradescope. $\text{\LaTeX}$ ¹ is preferred but not required. If you use another editor for your solutions (such as Microsoft Word), you should convert the final document to a PDF, confirm that the symbolic math from the equation editor is properly formatted, and submit the PDF. You must mark the locations of your solutions to individual problems on Gradescope as explained in [the documentation](#). Implementation problems will request that you submit code separately on Gradescope to be autograded.

Writing Expectations. If you are asked to provide an algorithm, you should clearly and unambiguously define every step of the procedure as a combination of precise sentences in plain English or pseudocode. If you are asked to explain your algorithm, its runtime complexity, or argue for its correctness, your written answers should be clear, concise, and should show your work. Do not skip details but do not write paragraphs where a sentence suffices. Results covered in lecture may be referenced and used directly without any justification of their correctness (unless asked for).

Collaboration. You can (and should!) work with a partner, in which case you will submit a single solution as a group on Gradescope *by adding your partner to your submission on Gradescope* ([see here for directions](#)). *Always cite anyone/anything consulted* outside of course materials, including a brief description of their contribution to your work. See also our policy on collaboration on the [course webpage](#).

Grading. Theory problems will be graded by TAs on an S/U scale (for each sub-problem). Applied problems typically have a separate autograder where you can see your score. **The lowest scoring problem is dropped.** See the [course assignments webpage](#) for more grading details.

¹If you are new to $\text{\LaTeX}$, you can download it for free at [latex-project.org](https://www.latex-project.org) or you can use the popular and free (for a personal account) cloud-editor [overleaf.com](https://www.overleaf.com). We also recommend [overleaf.com/learn](https://www.overleaf.com/learn) for tutorials and reference.

Problem 1

Consider the following algorithm `SILLYSORT` and its subroutine `SMERGE`. Aptly named, `SILLYSORT` is a MergeSort-like algorithm that sorts the given array $R[1..n]$ of n integers. For all parts, assume that n is a power of two and R contains distinct integers (no duplicates).

```
SILLYSORT( $R[1..n]$ ):
  if  $n > 1$ :
    SILLYSORT( $R[1..n/2]$ )
    SILLYSORT( $R[n/2 + 1..n]$ )
    SMERGE( $R[1..n]$ )
```

```
SMERGE( $R[1..n]$ ):
  if  $n = 2$ :
    if  $R[1] > R[2]$ :
      swap  $R[1] \longleftrightarrow R[2]$ 
  else:
    for  $i \leftarrow 1$  to  $n/4$ :
      swap  $R[i + n/4] \longleftrightarrow R[i + n/2]$     «swap middle fourths»
    SMERGE( $R[1..n/2]$ )    «leftmost half»
    SMERGE( $R[n/2 + 1..n]$ )    «rightmost half»
    SMERGE( $R[n/4 + 1..3n/4]$ )    «center half»
```

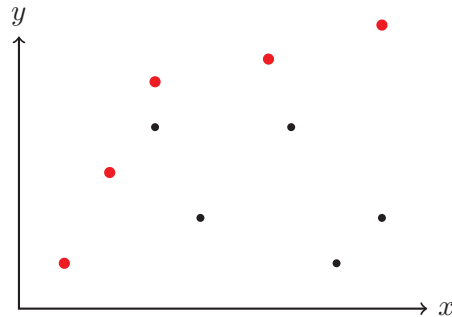
Note: The notation $R[i..j]$ is a pointer to the subarray of an array R from indices i to j (inclusive). Any operations on a subarray are in-place and thus occur on the original array that contains it. (This is the same as for the description of `MERGE Sort` and `QUICK Sort` from our textbook/lecture.)

- (a) Use induction to prove that `SmERGE( $R[1..n]$ )` sorts the given array R in-place, assuming that $R[1..n/2]$ and $R[n/2 + 1..n]$ are sorted. *[Hint: It may be useful to first follow the smallest $n/4$ and largest $n/4$ elements in R before/between/after the initial for-loop and recursive calls.]*
- (b) State a recurrence that describes the runtime of `SmERGE`, then solve it using the recursion tree method.
- (c) State a recurrence that describes the runtime of `SillySort`, then solve it using the recursion tree method.

Additional ungraded exercise—do not submit: Prove that `SILLYSORT( $R[1..n]$ )` indeed sorts R using induction, assuming the correctness of `SmERGE` from part (a).

Problem 2

Let $S = \{p_i = (x_i, y_i) : 1 \leq i \leq n\}$ be a set of n points on a 2D plane. We say that a point p_i is *northwest* (NW) of p_j if $x_i \leq x_j$ and $y_i \geq y_j$, treating the $(-x)$ -direction (resp., $(+y)$ -direction) as west (resp., north). A point p_i is a *northwesterly point* of S if there are no other points in S NW of p_i . In the example below, the larger red points are the northwesterly points of S .



Describe an algorithm that returns the northwesterly points of S in $O(n \log n)$ time. Prove the correctness of your algorithm using induction, then state and justify its runtime using the recursion tree method. [Hint: Sort then divide S into two sets based on the x -coordinates of points, and “conquer.” What is the appropriate merge/combine step to obtain the overall solution?]

Problem 3

An array $A[1..n]$ of n distinct integers (no duplicates) is considered *twirled* if it consists of two non-empty contiguous sequences, L and R , that are sorted in increasing order and all elements in L are greater than those in R . In other words, if R instead appeared left of L , the entire resulting sequence would be sorted in increasing order.

For example, in the following twirled array, the left subsequence is underlined and has length 6, the right subsequence is overlined and has length 4, and indeed all elements in the left subsequence are greater than those in the right subsequence:

$[\underline{12, 14, 19, 23, 25, 27}, \overline{2, 3, 5, 7}]$

Describe an $O(\log n)$ -time algorithm to return the length of the left subsequence of a given twirled array A . Prove its correctness using induction, then state and justify its runtime using the recursion tree method.

[Hint: Consider a variant of binary search. Given an element x of A , what information do you need to decide which of the left and right subsequences that x belongs to?]

Problem 4

In the following, all arrays/sequences are 0-indexed (as in Python). A *flippable list* is a data structure f which stores a finite sequence S_f of distinct integers and supports the following two operations:

- $\text{PEEK}(i)$: Return the object at index i in S_f , where 0 is the first index.
- $\text{FLIP}(i, j)$: Reverse (“flip”) the subsequence $S_f[i..j-1]$ if $i < j-1$ and do nothing otherwise. (NOTE: Notice the $j-1$ in the definition. This is to mimic Python’s syntax: `x[i:j]` returns the sublist of list `x` from `i` to `j-1`.)

Example: If $S_f = [4, 5, 2, 1, 3]$, $\text{PEEK}(1)$ returns 5 and $\text{FLIP}(1, 4)$ modifies S_f so that it becomes $[4, \underline{1}, \underline{2}, \underline{5}, 3]$, where the underlined symbols are those in the affected subsequence.

For this problem, you will complete an implementation of a method similar to the `PARTITION` subroutine for `QUICKSORT` in Python 3. Download the class file `flippable_list.py` and your template file `flartition.py` on Canvas in the Files section. The file `flippable_list.py` includes the `FlippableList` class, which supports the `PEEK` and `FLIP` operations. For any Python list S , a `FlippableList` object representing S is constructed and returned by calling `FLIPPABLELIST(S)`. **You may not directly access the `_lst` attribute of the `FlippableList` objects in your submission, otherwise you will receive a grade of 0.** However, you may find checking it to be useful while testing your code. See also the helper functions provided in the file.

`flartition.py` is the file in which you will complete the following task and cite any/all collaboration before uploading it to Gradescope under “HW1 Problem 4 (Code)”. This problem has no written component.

Let f be a flippable list, let i, j be indices of S_f , and let p be an integer (the “pivot” value). Design and implement a recursive method `FLARTITION( $f, i, j, p$ )` that modifies the subsequence $S_f[i..j-1]$, using only `PEEK` and `FLIP` operations on f , and returns an index k where:

1. all elements in $S_f[i..k-1]$ are all *less than* p , and
2. all elements in $S_f[k..j]$ are all *at least* p .

In other words, the returned index k is the first index of an element in the modified subsequence that is at least p ; if all elements in the subsequence are less than p , then $k = j$.²

Example: For flippable list f with sequence $S_f = [4, 9, 330, 1, 2, 5, 8, 16]$, a valid value of S_f after `FLARTITION( $f, 2, 7, 6$ )` is $[4, 9, \underline{2}, \underline{1}, \underline{5}, \underline{330}, 8, 16]$, where the underlined elements are those in the affected subsequence. The return value of the call is 5, which is the index of the first element (330) at least the pivot value of 6. The orderings of the elements less than/at least the pivot value 6 are arbitrary.

[Hint: Split the problem into two halves, conquer, and then finish. Note that the median index of a subsequence $S_f[i..j-1]$ can be computed as $(i+j)/2$, rounded down.]

²Note that this method differs from the `PARTITION` subroutine from `QUICKSORT` in the notes/lecture. In particular, the parameter p need not be in $S_f[i..j-1]$, and elements equal to p may end up anywhere after the elements smaller than p , not necessarily before those greater than p .